

SQL - Consultas

Disciplina: Banco de Dados

Prof. Tiago Piperno Bonetti

bonetti@prof.unipar.br

Material adaptado dos slides dos livros: Projeto de Banco de Dados e Banco de Dados Relacional, Heuser, Carlos A.

Consultas

Parte de SQL referente à **manipulação do conteúdo** do banco de dados (DML).

O comando **SELECT** é utilizado para escrever consultas ao banco de dados.

Ele é central na linguagem, sendo algumas de suas cláusulas compartilhadas com outros comandos.

Sua sintaxe é vasta, com um grande número de alternativas.

Além de outras, o comando implementa as operações das linguagens teóricas de consulta, **álgebra e cálculo relacional**.

Expressões

Expressões são centrais no comando SELECT

Há dois tipos:

- Expressão de **valor**
 - Retorna um valor escalar (número, cadeia de caracteres,...)
- Expressão de **tabela**
 - Retorna uma tabela

Expressões

expressãoValor

constante
| referenciaColuna
| expressãoValor { + | - | * | / | ** } expressãoValor
| expressãoValor || expressãoValor
| UPPER (expressãoValor)
| LOWER (expressãoValor)
| expressãoLógica
| (expressãoValor)
| (subConsultaEscalar)

referenciaColuna

[nomeTabela].nomeColuna

Expressões

```
1  /* === constantes === */
2  125                /* número inteiro */
3  54.454             /* número decimal */
4  'Rio'              /* cadeia de caracteres */
5  '2000-02-30'       /* data */
6  NULL               /* vazio */
7
8  /* === referências à coluna === */
9  nome_peca
10 peca.nome_peca
```

Expressões

```
12  /* === expressões numéricas === */
13  5 * ( 12 + peso_peca )
14  peso_peca - 12.5
15
16  /* === expressões sobre
17           cadeias de caracteres === */
18  cidade || ' - RS' /* concatenação */
19  /* funções */
20  UPPER(cidade) /* converte para maiúsculas */
21  LOWER(cidade) /* converte para minúsculas */
```

A lógica de três valores

Uma coluna de uma tabela pode ser definida como opcional

- Significa que ela pode conter o **valor vazio** (null)

Interpretação de vazio é feita pela aplicação:

- valor desconhecido,
- valor não se aplica,
- valor propositalmente omitido,
- ...

A lógica de três valores

A abordagem relacional usa uma lógica de três valores

Uma condição **resulta** em um de três valores:

- Falso (**FALSE**)
- Verdadeiro (**TRUE**)
- Desconhecido (**UNKNOWN**)

A lógica de três valores

Toda **comparação** com um valor **vazio resulta** em desconhecido (**UNKWON**)

Exemplo: Se um campo **nome** for vazio, a comparação

- **nome** = 'João'

Resulta em **desconhecido (UNKWON)**.

Ideia por trás:

- Se o campo está vazio, o SGBD não sabe se o valor é ou não igual à cadeia de caracteres.
- Não é possível determinar se o resultado da comparação é falso ou verdadeiro.

Expressão Lógica

expressãoLógica

TRUE

| FALSE

| UNKNOWN

| expressãoValor { = | <> | != | < | <= | > | >= | LIKE }

| expressãoValor

| expressãoValor IS NULL

| expressãoValor IS NOT NULL

| NOT expressãoLógica

| expressãoLógica OR expressãoLógica

| expressãoLógica AND expressãoLógica

| expressãoLógicaExists

| expressãoLógicaIN

Expressão Lógica

```
1  /* === constantes === */
2  TRUE
3  FALSE
4  UNKNOWN
5
6  /* === comparações de valores === */
7  5 = peso_pecas
8  peso_pecas <> 12.5
9  peso_pecas != 12.5
10 cidade_pecas LIKE 'Ri%'
11
12 /* === testes por vazio === */
13 cidade_pecas IS NULL
14 cidade_pecas IS NOT NULL
15
16 /* === conectores lógicos === */
17 cidade_pecas != 'Rio' AND peso_pecas = 10
18 NOT cidade_pecas = 'Rio'
```

Comando SELECT - sintaxe

comandoSelect

SELECT

[ALL | DISTINCT] { * | colunaSelect [, ...] }

FROM tabelaSelect [, ...]

[WHERE expressãoLógica]

[GROUP BY expressãoValor [, ...]]

[HAVING expressãoLógica]

[{ UNION | INTERSECT | EXCEPT } [ALL]

comandoSelect

]

[ORDER BY { expressãoValor [ASC | DESC] } [, ...]]

A sintaxe do comando SELECT é vasta, já no SQL padrão.

Uma versão reduzida do comando, suficiente para permitir a compreensão geral de seu funcionamento bem como sua relação com as linguagens teóricas de consulta.

Comando SELECT - sintaxe

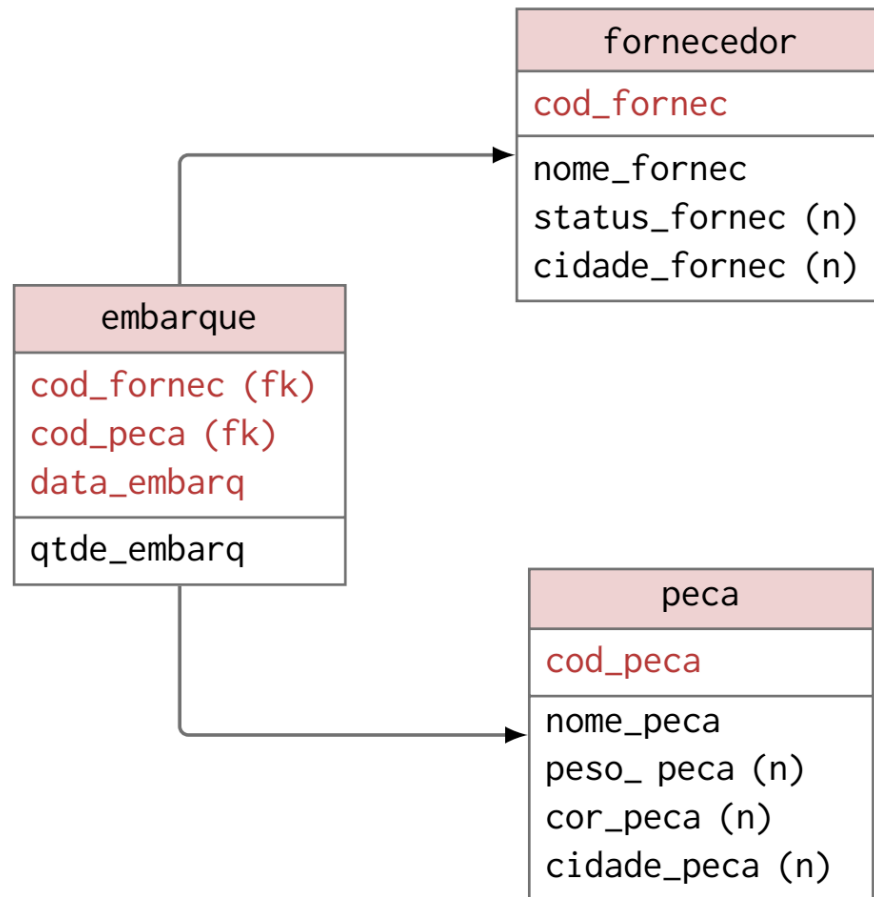
comandoSelect

SELECT

```
[ ALL | DISTINCT ] { * | colunaSelect [ , ... ] }  
FROM tabelaSelect [ , ... ]  
[ WHERE expressãoLógica ]  
[ GROUP BY expressãoValor [ , ... ] ]  
[ HAVING expressãoLógica ]  
[ { UNION | INTERSECT | EXCEPT } [ ALL ]  
  comandoSelect  
]  
[ ORDER BY { expressãoValor [ ASC | DESC ] } [ , ... ] ]
```

Vamos por
partes!

Comando SELECT



Comando SELECT

```
SELECT *  
FROM peca
```

peca

cod_peca	nome_peca	peso_peca	cor_peca	cidade_peca
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>

Comando SELECT

```
SELECT *  
FROM peca
```

Obter todos os dados de todas as peças

peca

cod_peca	nome_peca	peso_peca	cor_peca	cidade_peca
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>

Comando SELECT

```
SELECT *  
FROM peca  
WHERE cidade_pecas = 'Rio'
```

peca

cod_pecas	nome_pecas	peso_pecas	cor_pecas	cidade_pecas
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>

Comando SELECT

```
SELECT *  
FROM peca  
WHERE cidade_pecas = 'Rio'
```

Obter todos os dados das peças que são da cidade 'Rio'

peca

cod_pecas	nome_pecas	peso_pecas	cor_pecas	cidade_pecas
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>

Comando SELECT

```
SELECT
    cod_pecas ,
    nome_pecas ,
    peso_pecas
FROM   pecas
WHERE  cidade_pecas = 'Rio'
```

pecas

cod_pecas	nome_pecas	peso_pecas	cor_pecas	cidade_pecas
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>

Comando SELECT

```
SELECT
    cod_pecas ,
    nome_pecas ,
    peso_pecas
FROM   pecas
WHERE  cidade_pecas = 'Rio'
```

Obter o código, o nome e o peso de
cada peça cuja cidade é 'Rio'

pecas

cod_pecas	nome_pecas	peso_pecas	cor_pecas	cidade_pecas
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>

Comando SELECT

```
SELECT *  
FROM peca  
WHERE ( cidade_pecas = 'Rio' AND  
        peso_pecas > 15  
      )
```

peca

cod_pecas	nome_pecas	peso_pecas	cor_pecas	cidade_pecas
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>

Comando SELECT

```
SELECT *  
FROM peca  
WHERE ( cidade_pecas = 'Rio' AND  
        peso_pecas > 15  
      )
```

Obter as peças que são da cidade denominada 'Rio' e cujo peso excede 15

peca

cod_pecas	nome_pecas	peso_pecas	cor_pecas	cidade_pecas
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>

Comando SELECT

```
SELECT
    peca.cod_pecas ,
    cod_fornec ,
    data_embarq
FROM    peca ,
        embarque
WHERE   peca.cod_pecas = embarque.cod_pecas
AND     peca.nome_pecas = 'Parafuso'
```

embarque

cod_fornec	cod_pecas	data_embarq	qtde_embarq
F1	P1	2000-01-12	300
F1	P1	2000-01-15	200
F1	P2	2000-01-12	350
F1	P3	2000-07-22	250
F1	P4	2000-01-12	150
F1	P5	2000-05-14	200
F2	P1	2000-01-12	300
F2	P1	2000-12-04	300
F2	P2	2000-12-04	350
F2	P3	2000-12-04	250
F2	P4	2000-09-24	150
F3	P2	2000-04-04	200
F3	P3	2000-10-30	350

pecas

cod_pecas	nome_pecas	peso_pecas	cor_pecas	cidade_pecas
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>

Comando SELECT

```
SELECT
    peca.cod_pecas ,
    cod_fornec ,
    data_embarq
FROM peca ,
     embarque
WHERE peca.cod_pecas = embarque.cod_pecas
AND peca.nome_pecas = 'Parafuso'
```

Para cada embarque de uma peça denominada 'Parafuso', obter o código da peça, o código do fornecedor e a data do embarque

embarque

cod_fornec	cod_pecas	data_embarq	qtde_embarq
F1	P1	2000-01-12	300
F1	P1	2000-01-15	200
F1	P2	2000-01-12	350
F1	P3	2000-07-22	250
F1	P4	2000-01-12	150
F1	P5	2000-05-14	200
F2	P1	2000-01-12	300
F2	P1	2000-12-04	300
F2	P2	2000-12-04	350
F2	P3	2000-12-04	250
F2	P4	2000-09-24	150
F3	P2	2000-04-04	200
F3	P3	2000-10-30	350

pecas

cod_pecas	nome_pecas	peso_pecas	cor_pecas	cidade_pecas
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>


```

SELECT
    peca.cod_pecas ,
    embarque.cod_fornec ,
    nome_fornec ,
    data_embarq
FROM   peca ,
       embarque ,
       fornecedor
WHERE  peca.cod_pecas = embarque.cod_pecas
AND    fornecedor.cod_fornec =
        embarque.cod_fornec
AND    nome_pecas = 'Parafuso'

```

embarque			
cod_fornec	cod_pecas	data_embarq	qtde_embarq
F1	P1	2000-01-12	300
F1	P1	2000-01-15	200
F1	P2	2000-01-12	350
F1	P3	2000-07-22	250
F1	P4	2000-01-12	150
F1	P5	2000-05-14	200
F2	P1	2000-01-12	300
F2	P1	2000-12-04	300
F2	P2	2000-12-04	350
F2	P3	2000-12-04	250
F2	P4	2000-09-24	150
F3	P2	2000-04-04	200
F3	P3	2000-10-30	350

peca				
cod_pecas	nome_pecas	peso_pecas	cor_pecas	cidade_pecas
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>

fornecedor			
cod_fornec	nome_fornec	status_fornec	cidade_fornec
F1	Antunes	5	Porto Alegre
F2	Silva	10	Porto Alegre
F3	Souza	15	Curitiba
F6	Antunes	10	Rio
F4	Machado	10	<N>
F5	Barcelos	12	Rio

```

SELECT
    peca.cod_pecas ,
    embarque.cod_fornec ,
    nome_fornec ,
    data_embarq
FROM   peca ,
       embarque ,
       fornecedor
WHERE  peca.cod_pecas = embarque.cod_pecas
AND    fornecedor.cod_fornec =
        embarque.cod_fornec
AND    nome_pecas = 'Parafuso'

```

Para cada embarque de uma peça denominada 'Parafuso', obter o código da peça, o código do fornecedor, o nome do fornecedor, e a data do embarque.

peca.cod_pecas	embarque.cod_fornec	nome_fornec	data_embarq
P1	F1	Antunes	2000-01-12
P1	F1	Antunes	2000-01-15
P1	F2	Silva	2000-01-12
P1	F2	Silva	2000-12-04

embarque			
cod_fornec	cod_pecas	data_embarq	qtde_embarq
F1	P1	2000-01-12	300
F1	P1	2000-01-15	200
F1	P2	2000-01-12	350
F1	P3	2000-07-22	250
F1	P4	2000-01-12	150
F1	P5	2000-05-14	200
F2	P1	2000-01-12	300
F2	P1	2000-12-04	300
F2	P2	2000-12-04	350
F2	P3	2000-12-04	250
F2	P4	2000-09-24	150
F3	P2	2000-04-04	200
F3	P3	2000-10-30	350

peca				
cod_pecas	nome_pecas	peso_pecas	cor_pecas	cidade_pecas
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>

fornecedor			
cod_fornec	nome_fornec	status_fornec	cidade_fornec
F1	Antunes	5	Porto Alegre
F2	Silva	10	Porto Alegre
F3	Souza	15	Curitiba
F6	Antunes	10	Rio
F4	Machado	10	<N>
F5	Barcelos	12	Rio

Cadeias de caracteres

Comparação de cadeias é estrita:

- Maiúsculas diferente de minúsculas, ...
 - Tiago $\neq$ tiago

Funções como **UPPER** e **LOWER** podem ser usadas

Cadeias de caracteres

```
SELECT *  
FROM fornecedor  
WHERE cidade_fornec <> 'Porto Alegre '  
      OR cidade_fornec IS NULL
```

fornecedor

cod_fornec	nome_fornec	status_fornec	cidade_fornec
F1	Antunes	5	Porto Alegre
F2	Silva	10	Porto Alegre
F3	Souza	15	Curitiba
F6	Antunes	10	Rio
F4	Machado	10	<N>
F5	Barcelos	12	Rio

Cadeias de caracteres

```
SELECT *  
FROM fornecedor  
WHERE cidade_fornec <> 'Porto Alegre'  
OR cidade_fornec IS NULL
```

fornecedor

cod_fornec	nome_fornec	status_fornec	cidade_fornec
F1	Antunes	5	Porto Alegre
F2	Silva	10	Porto Alegre
F3	Souza	15	Curitiba
F6	Antunes	10	Rio
F4	Machado	10	<N>
F5	Barcelos	12	Rio

Obter os dados dos fornecedores cuja cidade é diferente de 'Porto Alegre', incluindo daqueles que não têm cidade informada.

***comparação estrita, logo "porto alegre" não vai para o resultado**

Cadeias de caracteres

```
SELECT *  
FROM peca  
WHERE UPPER(cidade_pecas) = 'RIO'
```

ou

```
SELECT *  
FROM peca  
WHERE LOWER(cidade_pecas) = 'rio'
```

fornecedor

cod_fornec	nome_fornec	status_fornec	cidade_fornec
F1	Antunes	5	Porto Alegre
F2	Silva	10	Porto Alegre
F3	Souza	15	Curitiba
F6	Antunes	10	Rio
F4	Machado	10	<N>
F5	Barcelos	12	Rio

Obter as peças que são da cidade denominada 'Rio'.

A palavra 'Rio' pode estar escrita em qualquer combinação de letras maiúsculas e minúsculas.

Casamento de padrões

Operador LIKE compara cadeia com o padrão.

cadeia LIKE padrão

Padrão:

- Sublinhado (_) serve como coringa para um caractere qualquer
- Símbolo de porcentagem (%) serve como coringa para uma cadeia qualquer

LIKE - exemplos

Comparações que resultam em **verdadeiro**

- 'Pedro' LIKE 'Pedro'
- 'Pedro' LIKE '_edro'
- 'Pedro' LIKE 'P%o'

Comparações que resultam em **falso**

- 'Pedro' LIKE 'Ped'
- 'Pedro' LIKE 'P_o'

LIKE - exemplos

```
SELECT
    cod_fornec ,
    nome_fornec
FROM   fornecedor
WHERE  nome_fornec LIKE 'S%'
```

fornecedor

cod_fornec	nome_fornec	status_fornec	cidade_fornec
F1	Antunes	5	Porto Alegre
F2	Silva	10	Porto Alegre
F3	Souza	15	Curitiba
F6	Antunes	10	Rio
F4	Machado	10	<N>
F5	Barcelos	12	Rio

LIKE - exemplos

```
SELECT
    cod_fornec ,
    nome_fornec
FROM    fornecedor
WHERE   nome_fornec LIKE 'S%'
```

fornecedor

cod_fornec	nome_fornec	status_fornec	cidade_fornec
F1	Antunes	5	Porto Alegre
F2	Silva	10	Porto Alegre
F3	Souza	15	Curitiba
F6	Antunes	10	Rio
F4	Machado	10	<N>
F5	Barcelos	12	Rio

**Obter o código e o nome de
todos fornecedores cujo nome
começa com a letra 'S'
maiúscula**

Tratamento de duplicatas

Em SQL (ao contrário da abordagem relacional teórica)

- O resultado de uma consulta **pode conter duplicatas**

Tratamento especificado pelas palavras-chave:

ALL (opção default)

- Especifica que o resultado é um multi-conjunto (pode conter linhas duplicadas)

DISTINCT

- Especifica que o resultado é um conjunto (duplicatas são eliminadas)

Tratamento de duplicatas

```
SELECT DISTINCT  
    cidade_fornec  
FROM fornecedor
```

fornecedor			
cod_fornec	nome_fornec	status_fornec	cidade_fornec
F1	Antunes	5	Porto Alegre
F2	Silva	10	Porto Alegre
F3	Souza	15	Curitiba
F6	Antunes	10	Rio
F4	Machado	10	<N>
F5	Barcelos	12	Rio

Tratamento de duplicatas

```
SELECT DISTINCT  
    cidade_fornec  
FROM fornecedor
```

cidade_fornec
Porto Alegre
Curitiba
Rio
<N>

fornecedor

cod_fornec	nome_fornec	status_fornec	cidade_fornec
F1	Antunes	5	Porto Alegre
F2	Silva	10	Porto Alegre
F3	Souza	15	Curitiba
F6	Antunes	10	Rio
F4	Machado	10	<N>
F5	Barcelos	12	Rio

**Obter as diferentes cidades em
que há fornecedores**

Tratamento de duplicatas

```
SELECT DISTINCT  
    cidade_fornec  
FROM fornecedor
```

cidade_fornec
Porto Alegre
Curitiba
Rio
<N>

fornecedor

cod_fornec	nome_fornec	status_fornec	cidade_fornec
F1	Antunes	5	Porto Alegre
F2	Silva	10	Porto Alegre
F3	Souza	15	Curitiba
F6	Antunes	10	Rio
F4	Machado	10	<N>
F5	Barcelos	12	Rio

Qual o resultado sem
DISTINCT



Tratamento de duplicatas

```
SELECT ALL  
    cidade_fornec  
FROM fornecedor
```

cidade_fornec
Porto Alegre
Porto Alegre
Curitiba
Rio
<N>
Rio

fornecedor

cod_fornec	nome_fornec	status_fornec	cidade_fornec
F1	Antunes	5	Porto Alegre
F2	Silva	10	Porto Alegre
F3	Souza	15	Curitiba
F6	Antunes	10	Rio
F4	Machado	10	<N>
F5	Barcelos	12	Rio

Obter as cidades em que há fornecedores. A cidade deve aparecer no resultado tantas vezes quantos fornecedores nela existirem.

Tratamento de duplicatas

```
SELECT DISTINCT
    cidade_fornec
FROM    fornecedor
WHERE   cidade_fornec IS NOT NULL
```

fornecedor

cod_fornec	nome_fornec	status_fornec	cidade_fornec
F1	Antunes	5	Porto Alegre
F2	Silva	10	Porto Alegre
F3	Souza	15	Curitiba
F6	Antunes	10	Rio
F4	Machado	10	<N>
F5	Barcelos	12	Rio

Tratamento de duplicatas

```
SELECT DISTINCT
    cidade_fornec
FROM    fornecedor
WHERE   cidade_fornec IS NOT NULL
```

Obter as diferentes cidades em que há fornecedores. Caso algum fornecedor não tenha cidade informada, o vazio não deve constar do resultado.

fornecedor

cod_fornec	nome_fornec	status_fornec	cidade_fornec
F1	Antunes	5	Porto Alegre
F2	Silva	10	Porto Alegre
F3	Souza	15	Curitiba
F6	Antunes	10	Rio
F4	Machado	10	<N>
F5	Barcelos	12	Rio

cidade_fornec
Porto Alegre
Curitiba
Rio

Eliminação de duplicatas e desempenho

Eliminação de duplicatas pode ser **custosa**:

- Custo de ordenação em memória externa ou equivalente

Só usar **DISTINCT** se **necessário**

Usar quando:

- Sem **DISTINCT** podem haver duplicatas no resultado
- A **eliminação** destas duplicatas é **requerida** pela aplicação

Tabelas e colunas no comando SELECT

- Atribuir **nomes** a **colunas** do resultado
- Definir **expressões** que computam os valores de colunas do resultado
- Atribuir **nomes** a **tabelas** que aparecem na consulta
- Definir **expressões** que computam **tabelas** a utilizar na consulta

Definição de tabela - sintaxe

tabelaSelect

```
{ nomeTabela | expressãoTabela }  
  [ [ AS ] nomeTabelaAlias ]  
| ( comandoSelect ) [ AS ] nomeTabelaAlias
```

Definição de tabela - sintaxe

tabelaSelect

```
{ nomeTabela | expressãoTabela }  
[ [ AS ] nomeTabelaAlias ]  
| ( comandoSelect ) [ AS ] nomeTabelaAlias
```

Nome (aliás) dado a uma tabela dentro de
uma consulta

-
palavra-chave AS é opcional

Definição de tabela - sintaxe

```
1 SELECT DISTINCT
2     p.cod_peca ,
3     nome_peca ,
4     peso_peca
5 FROM   peca AS p,
6        embarque e
7 WHERE  ( p.cod_peca = e.cod_peca AND
8         e.cod_fornec = 'F1'
9        )
```

Nome (**aliás**) usado para referir à
tabela peca dentro da consulta

Obter o código, o nome e o peso
de cada peça para a qual exista
ao menos um embarque do
fornecedor de código 'F1'

Definição de tabela - sintaxe

```
1 SELECT DISTINCT
2     p.cod_peca ,
3     nome_peca ,
4     peso_peca
5 FROM   peca AS p,
6        embarque e
7 WHERE  ( p.cod_peca = e.cod_peca AND
8         e.cod_fornec = 'F1'
9        )
```

Nome (**aliás**) usado para referir à
tabela peca dentro da consulta

Referência à tabela peca através
de seu aliás

Obter o código, o nome e o peso
de cada peça para a qual exista
ao menos um embarque do
fornecedor de código 'F1'

Definição de tabela - sintaxe

```
1 SELECT DISTINCT
2     p.cod_pecas ,
3     nome_pecas ,
4     peso_pecas
5 FROM   pecas AS p,
6     embarque e
7 WHERE  ( p.cod_pecas = e.cod_pecas AND
8         e.cod_fornec = 'F1'
9         )
```

Palavra-chave **AS** é opcional

Obter o código, o nome e o peso de cada peça para a qual exista ao menos um embarque do fornecedor de código 'F1'

Renomeação de coluna - sintaxe

```
SELECT cidade_fornec AS cidade  
FROM fornecedor
```

O nome (aliás) escolhido para a coluna irá aparecer no resultado da consulta

Coluna resultante de expressão

```
SELECT
    cod_pecas ,
    nome_pecas ,
    peso_pecas * 2.2046
        AS peso_libras_pecas
FROM peca
```

peca

cod_pecas	nome_pecas	peso_pecas	cor_pecas	cidade_pecas
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>

Coluna resultante de expressão

```
SELECT
    cod_pecas ,
    nome_pecas ,
    peso_pecas * 2.2046
        AS peso_libras_pecas
FROM pecas
```

Para cada peça, obter seu código, seu nome, e seu peso em libras.

O peso das peças no banco de dados encontra-se em quilogramas e o fator de conversão para libras é 2,2046.

A coluna contendo o peso da peça em libras deve ter o nome `peso_pecas_libras`.

pecas

cod_pecas	nome_pecas	peso_pecas	cor_pecas	cidade_pecas
P1	Parafuso	5	Cinza	Porto Alegre
P2	Arruela	5	Cinza	Porto Alegre
P3	Mancal	25	Vermelho	Rio
P4	Eixo	15	Verde	Rio
P5	Motor	65	Vermelho	<N>

cod_pecas	nome_pecas	peso_pecas	peso_libras_pecas
P1	Parafuso	5	11.023
P2	Arruela	5	11.023
P3	Mancal	25	55.115
....			

Subconsulta

Uma subconsulta é um comando **SELECT** usado **dentro** de um **outro** comando **SELECT** mais abrangente.

O termo `tabelaSelect`, através de sua segunda opção, permite que um outro comando **SELECT** (termo `comandoSelect`) seja usado para **definir** a **tabela sobre** a qual a **consulta opera**.

Subconsultas podem ser usadas para tornar uma consulta complexa **mais legível**.

Entretanto, há comandos de SQL, como o que será visto **mais adiante**, que **exigem** o **uso** de **subconsultas**, não sendo possível resolvê-los sem seu uso.

Subconsulta

```
1 SELECT
2     parafuso.cod_peca ,
3     embarque.cod_fornec ,
4     nome_fornec ,
5     data_embarq
6 FROM ( SELECT cod_peca
7         FROM peca
8         WHERE nome_peca = 'Parafuso'
9     ) AS parafuso ,
10    embarque ,
11    fornecedor
12 WHERE ( parafuso.cod_peca =
13         embarque.cod_peca AND
14         fornecedor.cod_fornec =
15         embarque.cod_fornec
16     )
```

Para cada embarque de uma peça denominada 'Parafuso', obter o código da peça, o código do fornecedor, o nome do fornecedor e a data do embarque

Subconsulta

```
1 SELECT
2     parafuso.cod_pecas ,
3     embarque.cod_fornec ,
4     nome_fornec ,
5     data_embarq
6 FROM ( SELECT cod_pecas
7         FROM pecas
8         WHERE nome_pecas = 'Parafuso'
9     ) AS parafuso ,
10    embarque ,
11    fornecedor
12 WHERE ( parafuso.cod_pecas =
13         embarque.cod_pecas AND
14         fornecedor.cod_fornec =
15         embarque.cod_fornec
16 )
```

Para cada embarque de uma peça denominada 'Parafuso', obter o código da peça, o código do fornecedor, o nome do fornecedor e a data do embarque

O resultado desta subconsulta é usado como operando da consulta mais abrangente

Subconsulta

```
1 SELECT
2     parafuso.cod_pecas ,
3     embarque.cod_fornec ,
4     nome_fornec ,
5     data_embarq
6 FROM ( SELECT cod_pecas
7         FROM pecas
8         WHERE nome_pecas = 'Parafuso'
9       ) AS parafuso ,
10 embarque ,
11 fornecedor
12 WHERE ( parafuso.cod_pecas =
13         embarque.cod_pecas AND
14         fornecedor.cod_fornec =
15         embarque.cod_fornec
16 )
```

Para cada embarque de uma peça denominada 'Parafuso', obter o código da peça, o código do fornecedor, o nome do fornecedor e a data do embarque

Nome dado ao resultado da subconsulta

Subconsulta

```
1 SELECT
2     parafuso.cod_pecas ,
3     embarque.cod_fornec ,
4     nome_fornec ,
5     data_embarq
6 FROM ( SELECT cod_pecas
7         FROM pecas
8         WHERE nome_pecas = 'Parafuso'
9       ) AS parafuso ,
10 embarque ,
11 fornecedor
12 WHERE ( parafuso.cod_pecas =
13         embarque.cod_pecas AND
14         fornecedor.cod_fornec =
15         embarque.cod_fornec
16 )
```

Para cada embarque de uma peça denominada 'Parafuso', obter o código da peça, o código do fornecedor, o nome do fornecedor e a data do embarque

Na consulta mais abrangente, resultado da subconsulta é usado como se fosse uma tabela do BD

Referências

Carlos A. Heuser. Banco de Dados Relacional.

Pichetti, Roni F.; Vida, Edinilson S.; Cortes, Vanessa S. M. P. Banco de Dados. Biblioteca virtual: <https://viewer.bibliotecaa.binpar.com/viewer/9786556900186/>

Heuser, Carlos A. Projeto de Banco de Dados - Volume 4 [Série Livros didáticos informática UFRGS]. Biblioteca virtual: <https://viewer.bibliotecaa.binpar.com/viewer/9788577804528/>